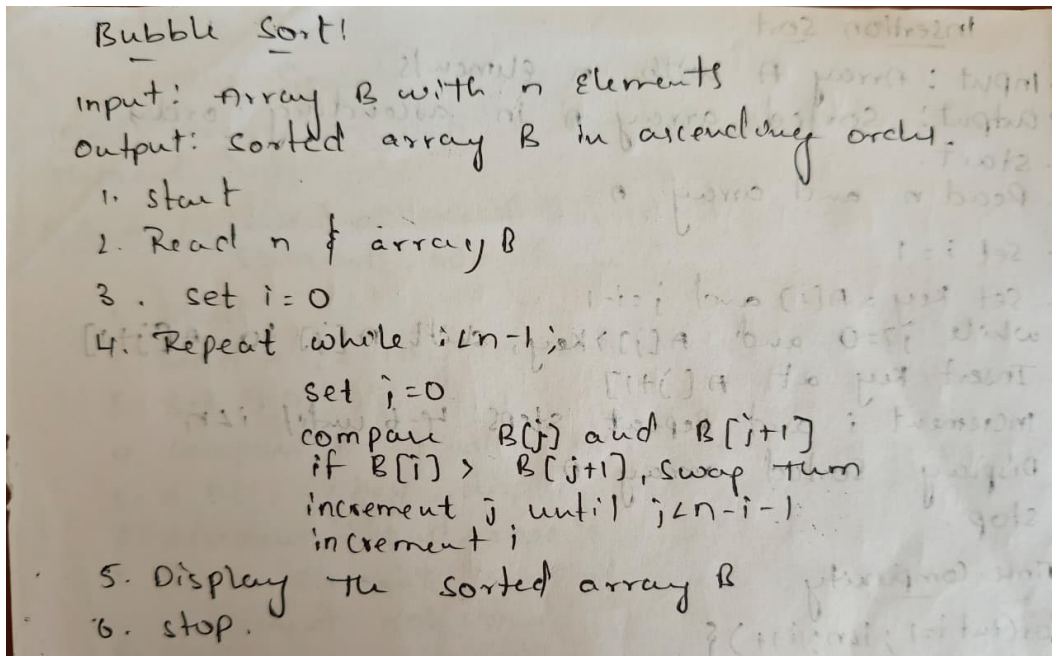


PRACTICAL: 01

**Aim: To implement and find the time analysis of sorting algorithms.
Bubble sort, Selection sort, Insertion sort, Merge sort and Quick sort.**

A) Bubble Sort:

Algorithm:



Program / Code:

```
#include<iostream>

using namespace std;

int main()
{
    int a[60], size, i, j, temp;

    cout<<"Enter the size of the array: ";

    cin>>size; cout<<"Enter the
    array elements: ";

    for(i=0; i<size; i++){
```



```
        cin>>a[i];
    }
    for(i=0; i<size-1; i++){
        for(j=0; j<size-i-1; j++){
            if(a[j]>a[j+1]){
                temp = a[j];
                a[j] = a[j+1];
                a[j+1] = temp;
            }
        }
    }
    cout<<"The Elements are: ";
    for(i=0; i<size; i++){
        cout<<a[i];
    }
    return 0;
}
```

Output:

```
Enter the size of array: 5
Enter Array Elements: 2 8 3 0 1
Before Sorting
2 8 3 0 1
After Sorting
0 1 2 3 8

=== Code Execution Successful ===
```

Time Complexity:

Time complexity: for (int i=0; i<n-1; i++) { } - outer loop
for (int j=0; j<(n-1-i); j++) { } - inner loop
if (arr[j] > arr[j+1]) { -1
int temp = arr[j]; -1
arr[j] = arr[j+1]; -1
arr[j+1] = temp; -1
}

Outer loop → Runs i=0 to n-1, so it executes n-1 times
Inner loop → pass(i) situations
0 n-1
1 n-2
2 n-3
3 n-4
4 n-5
5 n-6
6 n-7
7 n-8
8 n-9
9 n-10
10 n-11
11 n-12
12 n-13
13 n-14
14 n-15
15 n-16
16 n-17
17 n-18
18 n-19
19 n-20
20 n-21
21 n-22
22 n-23
23 n-24
24 n-25
25 n-26
26 n-27
27 n-28
28 n-29
29 n-30
30 n-31
31 n-32
32 n-33
33 n-34
34 n-35
35 n-36
36 n-37
37 n-38
38 n-39
39 n-40
40 n-41
41 n-42
42 n-43
43 n-44
44 n-45
45 n-46
46 n-47
47 n-48
48 n-49
49 n-50
50 n-51
51 n-52
52 n-53
53 n-54
54 n-55
55 n-56
56 n-57
57 n-58
58 n-59
59 n-60
60 n-61
61 n-62
62 n-63
63 n-64
64 n-65
65 n-66
66 n-67
67 n-68
68 n-69
69 n-70
70 n-71
71 n-72
72 n-73
73 n-74
74 n-75
75 n-76
76 n-77
77 n-78
78 n-79
79 n-80
80 n-81
81 n-82
82 n-83
83 n-84
84 n-85
85 n-86
86 n-87
87 n-88
88 n-89
89 n-90
90 n-91
91 n-92
92 n-93
93 n-94
94 n-95
95 n-96
96 n-97
97 n-98
98 n-99
99 n-100

$(n-1) + (n-2) + \dots + 2 + 1 = 1 + 2 + \dots + (n-1)$
 $T(n) = \frac{n(n-1)}{2} + n + 1$
 $T(n) = \frac{n^2 - n}{2} + n + 1$
 $T(n) = \frac{n^2 - n + 2n + 2}{2} = \frac{n^2 + n + 2}{2}$
 $T(n) = \frac{n^2}{2} + \frac{n}{2} + 1$
Worst case $T(n) = O(n^2)$
Avg case $T(n) = O(n^2)$
Best case $T(n) = O(n)$

Ignore all constants

B) Insertion Sort:

Algorithm:

Insertion Sort

- Input: Array A with n elements
- Output: Sorted array A in ascending order

- Start
- Read n and array A
- Set i = 1
- Set key = A[i] and j = i-1
- while j > 0 and A[j] > key, shift A[j] to A[j+1]
- Insert key at A[j+1]
- Increment i and Repeat steps 4-6 until i < n
- Display sorted array A
- Stop



Program / Code:

```
#include<iostream>

using namespace std;

int main(){

    int a[60], size, i, j, min, temp;

    cout<<"Enter the size of the array: ";

    cin>>size;

    cout<<"Enter the array elements: ";

    for(i=0; i<size; i++){

        cin>>a[i];

    }

    for(i=0; i<size-1; i++){

        min = i;

        for(j=i+1; j<size; j++){

            if(a[j] < a[min]){

                min = j;

            }

        }

        if(min != i){

            temp = a[i];

            a[i] = a[min];

            a[min] = temp;

        }

    }

    cout<<"The Elements are: ";

    for(i=0; i<size; i++){

        cout<<a[i]<<" ";

    }

}
```



```

    }

    return 0;
}

```

Output:

```

Enter the size of array: 5
Enter Array Elements: 3 9 0 5 1
Before Sorting
3 9 0 5 1
After Sorting
0 1 3 5 9

```

Time Complexity:

Time Complexity

```

for(int i=1; i<n; i++) {
    int key = arr[i];
    int j = i-1;
    while (j > 0 && arr[j] > key) {
        arr[j+1] = arr[j];
        j--;
    }
    arr[j+1] = key;
}

```

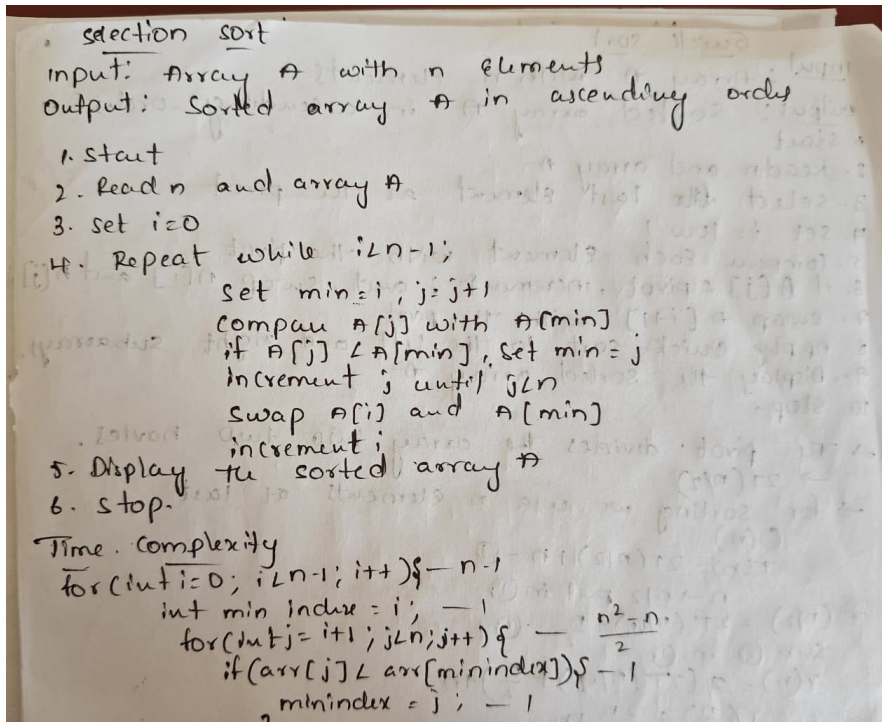
1. Best case: Already sorted array
 [10, 20, 30, 40, 50] → Outer loop runs $n-1$ times
 → while checks only once for each element
 $T(n) = O(n)$

2. Worst case: Reverse sorted array
 [50, 40, 30, 20, 10]
 Each element must be compared with all prev. elements and shifted
 pass
 1
 2
 3
 :
 n-1

comparision
 $1 + 2 + 3 + \dots + (n-1) + (n-2) + \dots + 1$
 $T(n) = \frac{n(n-1)}{2} = \frac{n^2 - n}{2}$
 $T(n) = O(n^2)$

C) Selection Sort:

Algorithm:



selection sort
input: Array A with n elements
output: Sorted array A in ascending order
1. Start
2. Read n and array A
3. set i=0
4. Repeat while i < n-1;
 Set min=i, j=j+1
 Compare A[j] with A[min]
 if A[j] < A[min], Set min=j
 Increment j until j < n
 Swap A[i] and A[min]
 Increment i
5. Display the sorted array A
6. Stop.
Time Complexity
for (int i=0; i < n-1; i++) { — n-1
 int minIndex = i; — 1
 for (int j=i+1; j < n; j++) { — $\frac{n^2-n}{2}$
 if (arr[j] < arr[minIndex]) { — 1
 minIndex = j; — 1
 }
 }
}

Program / Code:

```
#include <iostream>

using namespace std;

void selectionSort(int arr[], int n) {

    for (int i = 0; i < n - 1; i++) {

        int minIndex = i;

        for (int j = i + 1; j < n; j++) {

            if (arr[j] < arr[minIndex]) {

                minIndex = j;

            }

        }

        int temp = arr[minIndex];
```



```
arr[minIndex] = arr[i];  
arr[i] = temp;  
}  
}  
int main() {  
    int n; cout << "Enter number of  
    elements: "; cin >> n; int arr[n];  
    cout << "Enter " << n << "  
    elements: "; for (int i = 0; i < n;  
    i++) {  
  
        cin >> arr[i];  
    }  
    selectionSort(arr, n);  
  
    cout << "Sorted array: ";  
  
    for (int i = 0; i < n; i++){  
        cout << arr[i] << " ";  
    }  
  
    return 0;  
}
```

Output:

```
Enter the size of array: 5  
Enter Array Elements: 3 9 0 5 1  
Before Sorting  
3 9 0 5 1  
After Sorting  
0 1 3 5 9  
  
=== Code Execution Successful ===
```



Time complexity:

Time complexity

```

for (int i = 0; i < n-1; i++) {
    int minIndex = i;
    for (int j = i+1; j < n; j++) {
        if (arr[j] < arr[minIndex]) {
            minIndex = j;
        }
    }
    int temp = arr[minIndex];
    arr[minIndex] = arr[i];
    arr[i] = temp;
}

```

Outer loop: The outer loop runs from $i=0$ to $n-1$ so it executes $n-1$ times.

<u>inner loop</u>	pass	comparisons
1	$n-1$	$n-1$
2	$n-2$	$n-2$
3	$n-3$	$n-3$
$\vdots$	$\vdots$	$\vdots$
$n-1$	1	1

best case $\rightarrow O(n^2)$
avg case $\rightarrow O(n^2)$

$$T(n) = n-1 + 1 + \frac{n^2-n}{2} + 1 + 1 + 1$$

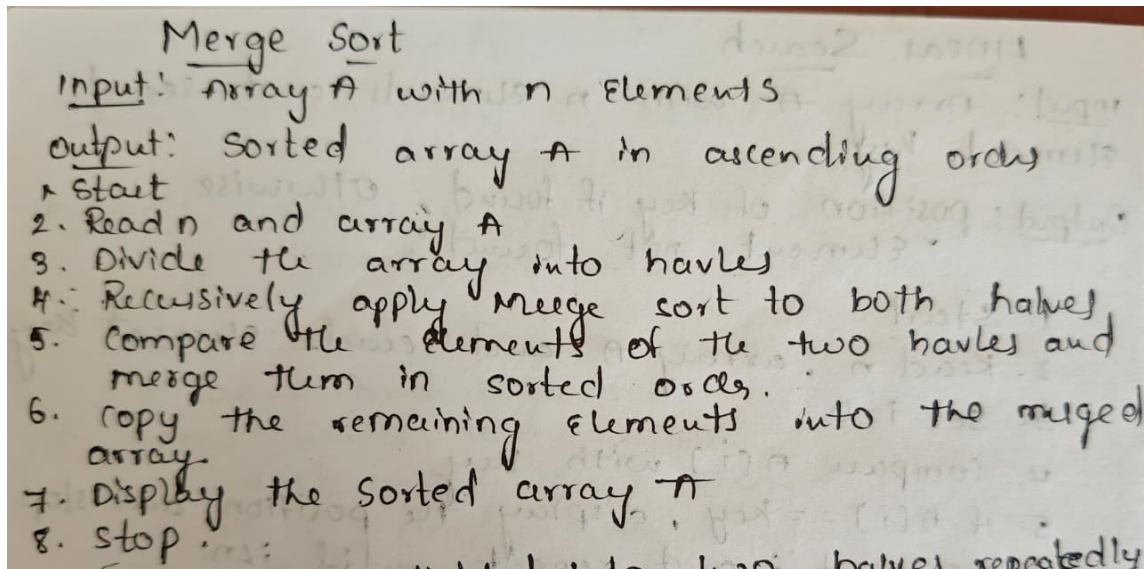
$$= n + \frac{n^2-n}{2} + 4 = \frac{2n + n^2 - n + 8}{2} = \frac{n^2 + n + 8}{2}$$

$$\approx \frac{n^2}{2} \Rightarrow O(n^2)$$

$\therefore O(n^2)$ = worst case

D) Merge Sort:

Algorithm:



Program / Code:

```
#include <iostream>
using namespace std;
void merge(int arr[], int left, int mid, int right) {
    int n1 = mid - left + 1;
    int n2 = right - mid;
    int L[n1], R[n2];
    for (int i = 0; i < n1; i++) {
        L[i] = arr[left + i];
```



```
    }  
    for (int j = 0; j < n2; j++) {  
        R[j] = arr[mid + 1 + j];  
    }  
    int i = 0;  
    int j = 0;  
    int k = left;  
    while (i < n1 && j < n2) {  
        if (L[i] <= R[j]) {  
            arr[k++] = L[i++];  
        } else {  
            arr[k++] = R[j++];  
        }  
    }  
    while (i < n1) {  
        arr[k++] = L[i++];  
    }  
    while (j < n2) {  
        arr[k++] = R[j++];  
    }  
}  
  
void mergeSort(int arr[], int left, int right) {  
    if (left < right) {  
        int mid = left + (right - left) / 2;  
        mergeSort(arr, left, mid);  
        mergeSort(arr, mid + 1, right);  
        merge(arr, left, mid, right);  
    }  
}
```

```
    }  
}  
  
int main() { int n; 11 cout << "Enter number of elements: ";  
  
    cin >> n;  
    int arr[n];  
    cout << "Enter elements: ";  
    for (int i = 0; i < n; i++) {  
        cin >> arr[i];  
    }  
    mergeSort(arr, 0, n - 1);  
    cout << "Sorted array using Merge Sort: ";  
    for (int i = 0; i < n; i++) {  
        cout << arr[i] << " ";  
    }  
    cout << endl;  
    return 0;  
}
```

Output:

```
Enter the size of array: 6  
Enter Array Elements: 2 8 0 1 5 3  
Before Sorting  
2 8 0 1 5 3  
After Sorting  
0 1 2 3 5 8  
  
=== Code Execution Successful ===
```



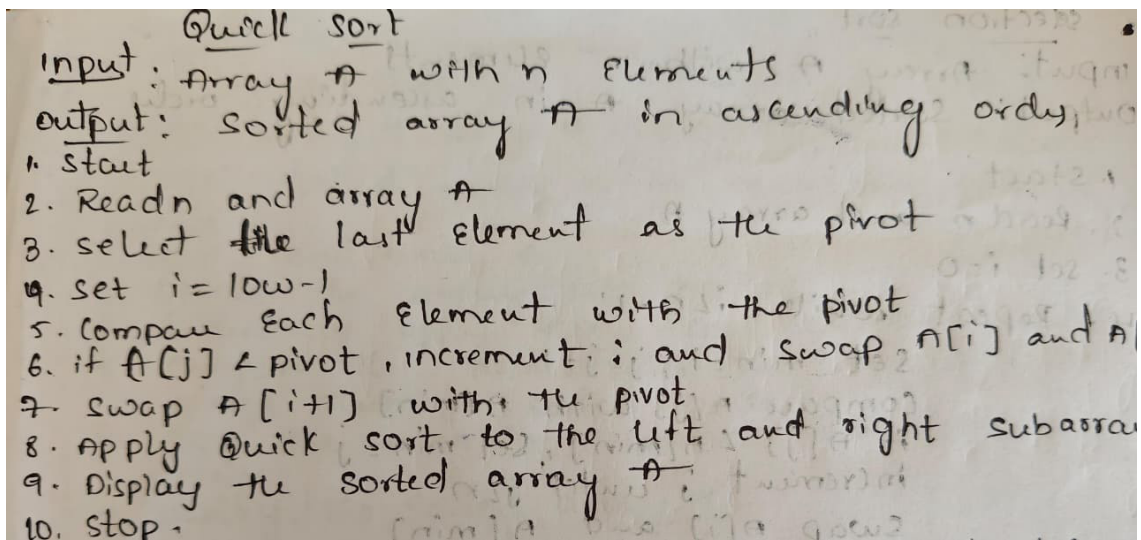
Time Complexity:

$\Rightarrow$ The array is divided into two halves repeatedly
 No. of divisions: $\log_2^n (2T(n/2))$
 merge: Each level merges all n elements. $O(n)$
 Total time $T(n) = 2T(n/2) + n$
 $T(n) = 2T(n/2) + n$ — (1)
 $n \rightarrow n/2$ put in (1)
 $T(n/2) = 2T(n/4) + n/2$
 $T(n/2) = 2T(n/4) + n/2$ — (2)
 Sub (2) in (1)
 $T(n) = 2^2T(n/4) + 2n$ — (3)
 $n \rightarrow n/4$ put in (3)
 $T(n/4) = 2^2T(n/8) + 2(n/4)$ — (4)
 Sub (4) in (3)
 $T(n) = 2^2[2^2T(n/8) + 2(n/4)] + 2n$
 $= 2^4T(n/8) + 4n$
 $T(n) = 2^kT(n/2^k) + kn$
 $T(n) = 2^kT(n/2^k) + kn$
 $\frac{n}{2^k} = 1$
 $n = 2^k$

Apply log both sides
 $\log n = \log 2^k$
 $k = \log n$
 $T(n) = 2^{\log n} + (\frac{n}{2} \log n) + \log n$
 $= n \log^2 + \frac{n}{2 \log^2} + \log n(n)$
 $= n \log n + n$
 $T(n) = O(n \log n)$
 Best case
 Avg case: $O(n \log n)$
 Worst case: $O(n \log n)$

E) Quick Sort:

Algorithm:



Quick Sort
Input: Array A with n elements
Output: Sorted array A in ascending order
1. Start
2. Read n and array A
3. select the last element as the pivot
4. set $i = \text{low} - 1$
5. Compare each element with the pivot
6. if $A[j] < \text{pivot}$, increment i and swap $A[i]$ and $A[j]$
7. swap $A[i+1]$ with the pivot
8. Apply Quick sort to the left and right subarray
9. Display the sorted array A
10. Stop

Program / Code:

```
#include <iostream>

using namespace std;

int partition(int arr[], int lb, int ub) {

    int pivot = arr[lb];

    int start = lb;
```




```
int end = ub;
while (start < end) {
    while (arr[start] <= pivot) {
        start++;
    }
    while (arr[end] > pivot) {
        end--;
    }
    if (start < end) {
        swap(arr[start], arr[end]);
    }
}
swap(arr[lb], arr[end]);
return end;
}

void quickSort(int arr[], int lb, int ub) {
    if (lb < ub) {
        int loc = partition(arr, lb,
            ub); quickSort(arr, lb, loc -
            1); quickSort(arr, loc + 1,
        } ub);
    }
}

int main() {
    int n;
    cout << "Enter size of array: ";
    cin >> n;
    int arr[n];
```



```
cout << "Enter " << n << " elements: ";  
for (int i = 0; i < n; i++) {  
    cin >> arr[i];  
}  
quickSort(arr, 0, n - 1);  
cout << "Sorted array using Quick Sort: ";  
for (int i = 0; i < n; i++) {  
    cout << arr[i] << " ";  
}  
return 0;  
}
```

Output:

```
Enter the size of array: 7  
Enter Array Elements: 9 0 3 7 1 5 6 2  
Before Sorting  
9 0 3 7 1 5 6  
After Sorting  
0 1 3 5 6 7 9  
  
=== Code Execution Successful ===
```

Time Complexity:

$\Rightarrow$ The pivot divides the array into two halves.
 $\rightarrow 2T(n/2)$
 $\rightarrow$ for sorting we use n elements at last.
 $O(n)$
 $T(n) = 2T(n/2) + n \quad \text{--- (1)}$
 $n \rightarrow n/2$ put in (1)
 $T(n/2) = 2T(n/4) + \frac{n}{2} \quad \text{--- (2)}$
 sub (2) in (1)
 $T(n) = 2(2T(n/4) + \frac{n}{2}) + n$
 $= 2^2T(\frac{n}{4}) + \frac{2n}{2} + n$
 $T(n) = 2^2T(\frac{n}{2^2}) + 2n \quad \text{--- (3)}$
 $n \rightarrow \frac{n}{4}$ put in (3)
 $T(n/4) = 2^2T(\frac{n}{2^4}) + 2(\frac{n}{4}) \quad \text{--- (4)}$
 sub (4) in (3)
 $T(n) = 2^2[2^2T(\frac{n}{2^4}) + \frac{2n}{4}] + 2n$
 $= 2^4T(\frac{n}{2^4}) + \frac{2^3n}{4} + 2n$
 $= 2^4T(\frac{n}{2^4}) + 4n$
 $T(n) = 2^kT(\frac{n}{2^k}) + kn$
 $\frac{n}{2^k} = 1, n = 2^k$
 apply $\log$ on b-s
 $\log n = \log 2^k$
 $k = \log n$
 $T(n) = 2^{\log n} T(\frac{n}{2^{\log n}}) + \log n(n)$
 $= n \log n + n$
 $T(n) = O(n \log n)$
 Best Case

Best case $O(n \log n)$
 Avg case $O(n \log n)$
 Worst case $O(n^2)$